

BN与神经网络调优

学习目标

神经网络优化

调参技巧

运行

批标准化 (Batch Normalization)

批标准化公式

过程图

为什么批标准化能够使优化过程变得简单

BN总结

学习目标

- 目标
 - 知道常用的一些神经网络超参数
 - 知道BN层的意义以及数学原理
- 应用
 - 无

神经网络优化

我们经常会涉及到参数的调优，也称之为超参数调优。目前我们从第二部分中讲过的超参数有

- 第法层面:
 - 学习率 α
 - $\beta_1, \beta_2, \epsilon$: Adam 优化简法的超参数,常设为 0.999、 10^{-8}
 - λ : 正则化网络参数,
- 网络层面:
 - hidden units: 各隐藏层神经元个数
 - layers: 神经网络层数

调参技巧

对于调参，通常采用跟机器学习中介绍的网格搜索一致，让所有参数的可能组合在一起，得到N组结果。然后去测试每一组的效果去选择。

假设我们现在有两个参数

α : 0.1, 0.01, 0.001, β : 0.8, 0.88, 0.9

这样会有9种组合, [0.1, 0.8], [0.1, 0.88], [0.1, 0.9].....

合理的参数设置：

- 学习率 α ：0.0001、0.001、0.01、0.1，跨度稍微大一些。
- 算法参数 β ，0.999、0.9995、0.998等，尽可能的选择接近于1的值

注：而指数移动平均值参数： β 从0.9（相当于近10天的影响）增加到0.9005对结果（ $1/(1-\beta)$ ）几乎没有影响，而 β 从0.999到0.9995对结果的影响会较大，因为是指数级增加。通过介绍过的式子理解

$$S_{100} = 0.1Y_{100} + 0.1 * 0.9Y_{99} + 0.1 * (0.9)^2Y_{98} + \dots$$

从相差大的来选择：学习率跨度比较大，beta参数就必须在1附近变换较小的值，因为指数增加原因

运行

通常我们有这么多参数组合，每一个组合运行训练都需要很长时间，但是如果资源允许的话，可以同时并行的训练多个参数模型，并观察效果。如果资源不允许的话。还是得一个模型一个模型的运行，并时刻观察损失的变化。

所以对于这么多的超参数，调优是一件复杂的事情，怎么让这么多的超参数范围，工作效果还能达到更好，训练变得更容易呢？

批标准化 (Batch Normalization)

Batch Normalization论文地址：

<https://arxiv.org/abs/1502.03167>

地址

训练深度神经网络很复杂，因为在训练期间每层输入分布发生变化，因为前一层的参数发生了变化。这通过要求较低的学习率和仔细的参数初始化来减慢训练速度，并且使得训练具有饱和和非线性的模型变得非常困难。我们将这种现象称为**内部

变量偏移**，并通过**标准化层**输入来解决问题。我们的方法的优势在于使标准化成为模型体系结构的一部分，并为

个培训小批量执行标准化。批量标准化允许我们使用更高的学习率并且不太关心初始化。它还可以充当调节器，在某些情况

下可以消除对Dropout的需求。应用于最先进的图像分类模型，批量标准化实现了相同的精度，培训步骤减少了14倍，并

且显著地超过了原始模型。使用批量标准化网络的集合，我们改进了ImageNet分类的最佳发布结果：达到4.9%的前5个

验证错误（和4.8%的测试错误），超出了人类评估者的准确性。

首先我们还是回到之前，我们对输入特征 X 使用了标准化处理。标准化后的优化得到了加速。对于深层网络呢？我们看以下公式，这是向量的表示，表示每Mini——batch有 m 个样本。

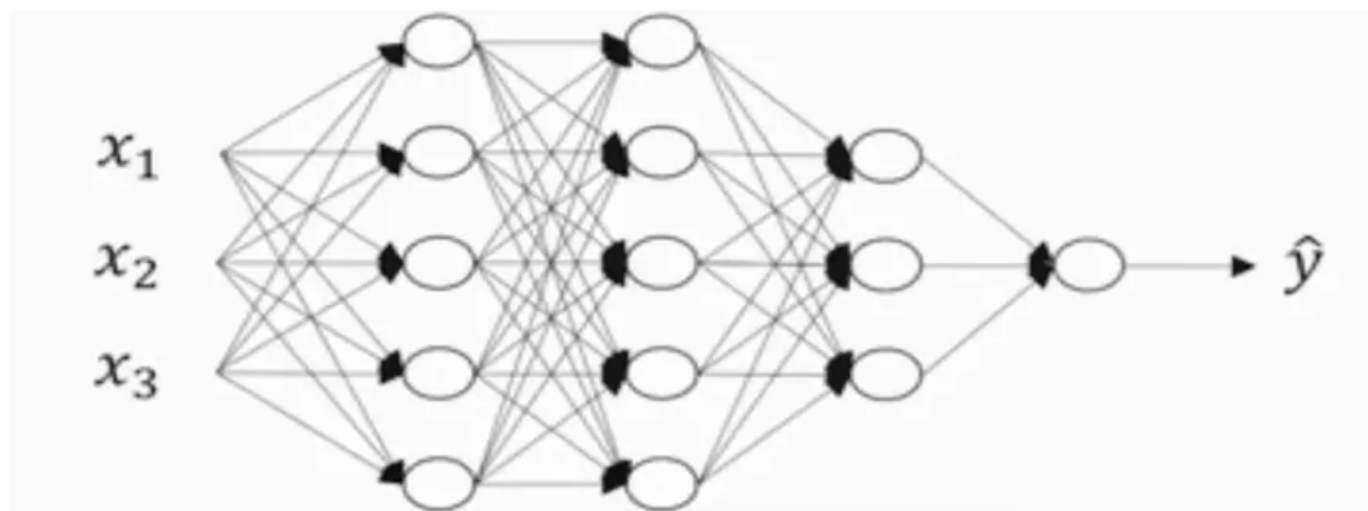
- m 个样本的向量表示

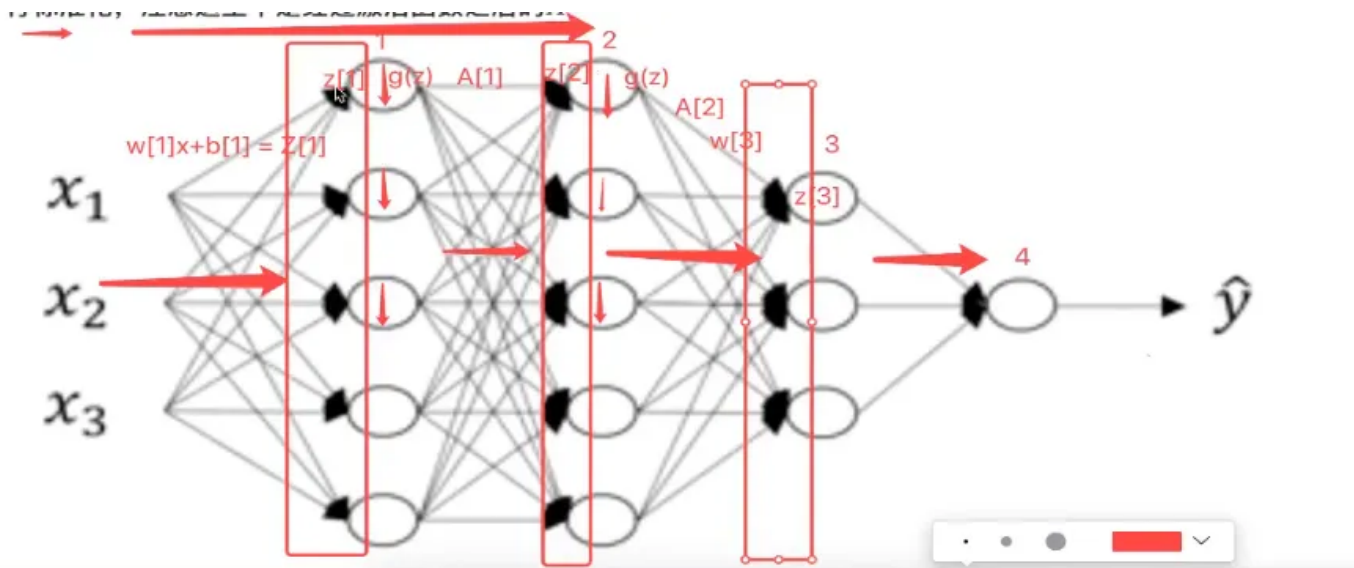
$$Z^{[L]} = W^{[L]} A^{[L-1]} + b^{[L]}$$

$$A^{[L]} = g^{[L]}(Z^{[L]})$$

输入 $A^{[L-1]}$, 输出 $A^{[L]}$

深层网络当中不止是初始的特征输入，而到了隐藏层也有输出结果，所以我们是否能够对隐层的输入 $Z^{[L]}$ 进行标准化，注意这里不是经过激活函数之后的 $A^{[L]}$





批标准化公式

所以假设对于上图第二个四个神经元隐层。记作 $z^{[l]}$,那么这一层会涉及多个 z ,所以我们默认用 $z_{[i]}^{[l]}$,为了简单显示去掉了 l 层这个标识,所以对于标准化中的平均值,以及方差

$$\mu = \frac{1}{m} \sum_i z^{(i)}$$

$$\sigma^2 = \frac{1}{m} \sum_i (z_i - \mu)^2$$

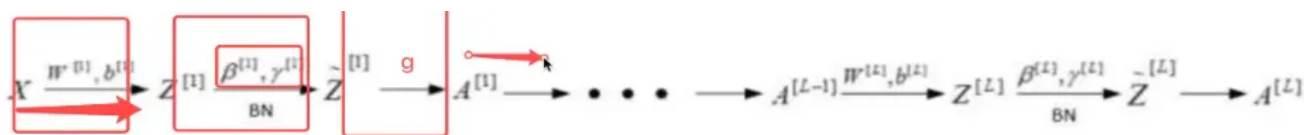
$$z_{norm} = \frac{z^{(i)} - \mu}{\sqrt{\sigma^2 + \epsilon}}$$

其中 ϵ 是为了防止分母为0,取值 10^{-8} 。这样使得所有的 l 层输入 $z_{[i]}^{[l]}$ 为0,方差为1。但是原文的作者不想让隐藏层单元总是含有平均值0和方差1,他认为也许隐藏层单元有了不同的分布会更有意义。因此,我们会增加这样的甲酸

$$\tilde{z}^{(i)} = \gamma z_{norm} + \beta$$

其中, γ 和 β 都是模型的学习参数(如同 W 和 b 一样),所以可以用各种梯度下降算法来更新 γ 和 β 的值,如同更新神经网络的权重一样。

过程图



每一层中都会有两个参数 β, γ 。

注：原论文的公式图

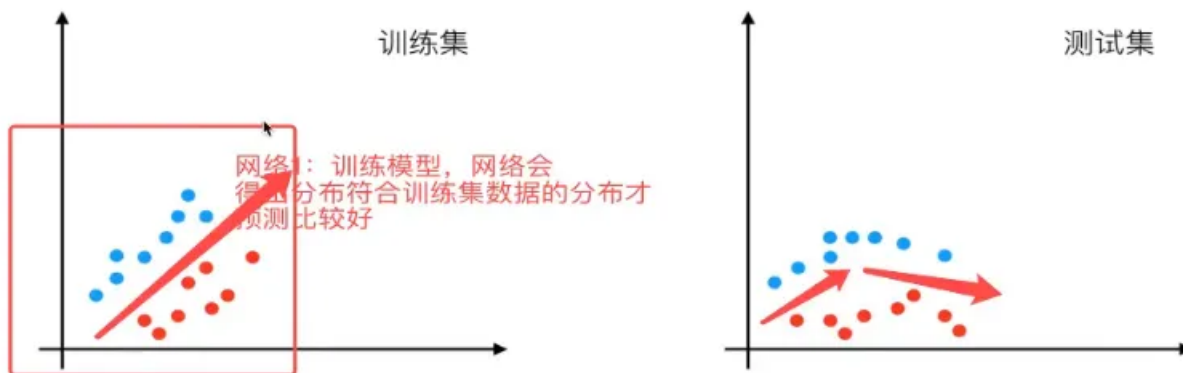
Input: Values of x over a mini-batch: $\mathcal{B} = \{x_1 \dots x_m\}$;
Parameters to be learned: γ, β

Output: $\{y_i = \text{BN}_{\gamma, \beta}(x_i)\}$

$$\mu_{\mathcal{B}} \leftarrow \frac{1}{m} \sum_{i=1}^m x_i \quad // \text{ mini-batch mean}$$
$$\sigma_{\mathcal{B}}^2 \leftarrow \frac{1}{m} \sum_{i=1}^m (x_i - \mu_{\mathcal{B}})^2 \quad // \text{ mini-batch variance}$$
$$\hat{x}_i \leftarrow \frac{x_i - \mu_{\mathcal{B}}}{\sqrt{\sigma_{\mathcal{B}}^2 + \epsilon}} \quad // \text{ normalize}$$
$$y_i \leftarrow \gamma \hat{x}_i + \beta \equiv \text{BN}_{\gamma, \beta}(x_i) \quad // \text{ scale and shift}$$

为什么批标准化能够使优化过程变得简单

我们之前在原文中标记了一个问题叫做叫做"internal covariate shift".这个词叫做叫做协变量偏移，但是不是很好理解。那么有一个解释叫做在网络当中数据的分布会随香不同数据集改变，这是网络中存在的问题。那我们一起来看一下数据本身分布是在这里会有什么问题。



也就是说如果我们训练集中的数据分布如左图，那么网络当中学习到的分布状况也就是左图。那对于给一个测试集中的数据，分布不一样。这个网络可能就不能准确去区分。这种情况下，一般要对模型进行重新训练。

Batch Normalization的作用就是减小**Internal Covariate Shift**所带来的影响，让模型变得更加健壮，鲁棒性（**Robustness**）更强。即使输入的值改变了，由于 **Batch Normalization** 的作用，使得均值和方差保持固定（由每一层 γ 和 β 决定），限制了在前层的参数更新对数值分布的影响程度，因此后层的学习变得更容易一些。**Batch Normalization** 减少了各层 W 和 b 之间的耦合性，让各层更加独立，实现自我训练学习的效果

BN总结

Batch Normalization 也起到微弱的正则化效果，但是不要将 Batch Normalization 作为正则化的手段，而是当作加速学习的方式。**Batch Normalization**主要解决的还是反向传播过程中的梯度问题（梯度消失和爆炸）。

○ 原理？

- 在线性计算之后，加入BN计算，4个步骤
 - 计算每一层，计算输出平均值
 - 计算每一层，标准差
 - 标准化结果
 - γ, β ，网络自动学习，每一层两个参数值不一样

